

深圳大学实验报告

课程名称：移动互联网应用

实验项目名称：我的校园

学院：计算机与软件学院

专业：软件工程

指导教师：解为成

报告人：学号：班级：

实验时间：2026年5月12日

实验报告提交时间：2026年5月27日

教务部制

实验目的与要求:

目的: 掌握安卓中活动的编写、自定义用户界面的开发、碎片开发、广播机制以及数据持久化技术等;并能通过对课堂知识进行扩展来完善该界面,并使界面尽量美观。

内容要求:

1. 请尽量模拟如下深大校园主页的功能,参考:

<https://www1.szu.edu.cn/>



2. 具体要求:

1) 该实现的界面在某些地方应体现出如下功能:

- 界面能对平板与手机平台进行自适应(参考第4章碎片);
- 能对用户身份有强制下线的功能,比如网络中断,登录界面强行退出并显示提示错误的界面;
- 界面某些地方体现数据持久化的技术,如文件数据的读取、存储的多种实现方式,并简单阐述几种实现方式具体的适用场景;
- 界面要比较工整,没必要实现参考界面上的所有子项,能保证自己的界面实现能有扩展到参考界面的能力即可。

2) 功能并不局限于上面的要求,可以根据自己的理解设计一些新的功能,并在报告文档中进行详细的阐述,作为报告的亮点;

3) APP 的布局尽快模仿参考界面,如果有较大的困难,可以只实现出右半边部分的界面,并尽量按上面要求进行完善;

4) 对于某一种功能,可以在不同的子项处采用多种实现方式,并比较这些实现方式的不同及优劣势。

5) 如果加入了基于深度学习的人工智能模块或功能,可以考虑加5分;

3. 参考: 尽量多的应用参考书《第一行代码 Android》第二版第2章活动、第3章 UI 开发第4章碎片、第5章广播机制与第6章数据持久化技术的各个知识点。

注意：

1. 实验报告中需要有功能的描述、实验结果的截屏图像及详细说明；
2. 也欢迎采用其它章节的知识点完成本次实验报告，如果实现的功能言之合理，会考虑酌情加分。

方法、步骤：

1.环境搭建与底层架构梳理：

配置 Android Studio 开发环境，确保引入 AppCompatActivity 等传统 UI 支持库。

编写基础架构类：创建 ActivityCollector（活动管理器）用于统一管理 Activity 的生命周期；创建 BaseActivity 作为所有活动的父类，便于统一注册全局系统级监听（如广播）。

2.UI 界面设计与模块划分：

分析深大电脑端主页结构，将其抽象为“左侧信息流”与“右侧功能矩阵”两大模块。

使用 Fragment（碎片）技术将这两个模块分别封装为 LeftFragment 和 RightFragment。

利用 XML 编写登录界面及各碎片的内部 UI，运用 LinearLayout、GridLayout 和 ScrollView 等容器确保界面工整美观，并大量使用深大红（#8A0025）作为应用主题色。

3.多平台自适应适配：

利用 Android 的限定符机制，分别建立 layout（手机视图）和 layout-sw600dp（平板/宽屏视图）。

在手机视图中引入底部导航栏实现 Fragment 切换；在平板视图中采用权重（layout_weight）布局实现左右双碎片同屏展示。

4.业务逻辑与数据持久化开发：

实现登录逻辑，并接入 SharedPreferences 实现密码记忆。

编写 MyDatabaseHelper 继承 SQLiteOpenHelper，初始化建表逻辑并插入深大办事大厅的预设数据。

在 Fragment 中实现 File 内部文件流读写，模拟缓存网络通知公告。

5.广播机制与异常处理逻辑接入：

在 BaseActivity 中动态注册自定义的强制下线广播接收器。

在界面中增加触发点（模拟网络中断），联调广播接收与 Activity 管理器的清栈动作。

实验过程及内容：

1.界面 UI 开发与多平台自适应

①右侧服务大厅 (RightFragment)采用外层包裹 ScrollView，内部按照业务模块（教师事务、学生事务、荔园生活、网上服务）使用多个 GridLayout 进行网格布局的方式。这使得功能入口排列整齐。

②自适应策略： 利用了 Fragment 的解耦特性。

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <!-- 上半部分：占满剩余空间的容器，用于加载 Fragment -->
    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1" />

    <!-- 分割线 -->
    <View
        android:layout_width="match_parent"
        android:layout_height="1dp"
        android:background="#CCCCCC"/>

    <!-- 底部导航栏 -->
    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="56dp"
        android:orientation="horizontal"
        android:background="#FFFFFF">

        <Button
            android:id="@+id/btn_nav_news"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="?android:attr/selectableItemBackground"
            android:text="首页新闻"
            android:textColor="#8A0025"
            android:textSize="16sp" />

        <View
            android:layout_width="1dp"
            android:layout_height="match_parent"
            android:background="#CCCCCC"
            android:layout_marginTop="10dp"
            android:layout_marginBottom="10dp"/>

        <Button
            android:id="@+id/btn_nav_services"
            android:layout_width="0dp"
            android:layout_height="match_parent"
            android:layout_weight="1"
            android:background="?android:attr/selectableItemBackground"
            android:text="服务大厅"
            android:textColor="#333333"
            android:textSize="16sp" />

    </LinearLayout>

</LinearLayout>
```

手机端 (res/layout/activity_main.xml): 采用上下结构, 主体为一个 FrameLayout 占位, 底部设有两个 Tab 按钮。用户点击按钮时, 通过 fragmentManager 动态 replace() 左右 Fragment。

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <!-- 左侧占用1份宽度 -->
    <fragment
        android:id="@+id/left_fragment"
        android:name="com.example.my_campus.mycampus.LeftFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="1" />

    <!-- 分割线 -->
    <View android:layout_width="1dp" android:layout_height="match_parent" android:background="#CCCCCC"/>

    <!-- 右侧占用2份宽度 (完全还原源大主页结构) -->
    <fragment
        android:id="@+id/right_fragment"
        android:name="com.example.my_campus.mycampus.RightFragment"
        android:layout_width="0dp"
        android:layout_height="match_parent"
        android:layout_weight="2" />

</LinearLayout>
```

平板端 (res/layout-sw600dp/activity_main.xml): 采用横向 LinearLayout, 直接将 LeftFragment 和 RightFragment 以 1:2 的空间权重静态写在布局中。当 App 在平板上运行时, 系统自动加载该布局, 完美还原 PC 端左右分栏的视觉效果。

2.全局广播与强制下线机制

①建立 ActivityCollector 集合, 重写所有 Activity 的 onCreate 和 onDestroy 将自身入栈出栈。

```
package com.example.my_campus.mycampus;

import android.app.Activity;
import java.util.ArrayList;
import java.util.List;

3 usages
public class ActivityCollector {
    4 usages
    public static List<Activity> activities = new ArrayList<>();

    1 usage
    public static void addActivity(Activity activity) {
        activities.add(activity);
    }

    1 usage
    public static void removeActivity(Activity activity) {
        activities.remove(activity);
    }

    1 usage
    public static void finishAll() {
        for (Activity activity : activities) {
            if (!activity.isFinishing()) {
                activity.finish();
            }
        }
        activities.clear();
    }
}
```

②在 BaseActivity 的 onResume 中动态注册了一个内部类 ForceOfflineReceiver（继承自 BroadcastReceiver），监听特定的 Action（如 com.szu.FORCE_OFFLINE）。

```
ActivityCollector.java x BaseActivity.java LoginActivity.java MyDatabaseHelper.java </> layout

1 package com.example.my_campus.mycampus;
2
3 import android.content.BroadcastReceiver;
4 import android.content.Context;
5 import android.content.Intent;
6 import android.content.IntentFilter;
7 import android.os.Bundle;
8 import androidx.appcompat.app.AlertDialog;
9 import androidx.appcompat.app.AppCompatActivity;
10
11 2 inheritors
12 public class BaseActivity extends AppCompatActivity {
13     5 usages
14     private ForceOfflineReceiver receiver;
15
16     2 overrides
17     @Override
18     protected void onCreate(Bundle savedInstanceState) {
19         super.onCreate(savedInstanceState);
20         ActivityCollector.addActivity(this);
21     }
22
23     @Override
24     protected void onResume() {
25         super.onResume();
26         IntentFilter intentFilter = new IntentFilter();
27         intentFilter.addAction("com.example.my_campus.FORCE_OFFLINE");
28         receiver = new ForceOfflineReceiver();
29         androidx.core.content.ContextCompat.registerReceiver(
30             context: this,
31             receiver,
32             intentFilter,
33             androidx.core.content.ContextCompat.RECEIVER_NOT_EXPORTED
34         );
35     }
36
37     @Override
38     protected void onPause() {
39         super.onPause();
40         if (receiver != null) {
41             unregisterReceiver(receiver);
42             receiver = null;
43         }
44     }
45
46     @Override
47     protected void onDestroy() {
48         super.onDestroy();
49         ActivityCollector.removeActivity(this);
50     }
51
52     // 内部类：强制下线广播接收器
53     static class ForceOfflineReceiver extends BroadcastReceiver {
54         @Override
55         public void onReceive(final Context context, Intent intent) {
56             AlertDialog.Builder builder = new AlertDialog.Builder(context);
57             builder.setTitle("警告");
58             builder.setMessage("您的账号在异地登录或网络异常，请重新登录！");
59             builder.setCancelable(false);
60             builder.setPositiveButton(text: "确定", (DialogInterface dialog, int which) -> {
61                 ActivityCollector.finishAll(); // 销毁所有活动
62                 Intent i = new Intent(context, LoginActivity.class);
63                 context.startActivity(i); // 重新启动登录界面
64             });
65             builder.show();
66         }
67     }
68 }
```

③在界面上设置一个“模拟网络异常”按钮，点击后通过 `sendBroadcast()` 发送该广播。接收器截获广播后，弹出一个不可取消的 `AlertDialog` 提示网络异常，用户点击确定后，调用 `ActivityCollector.finishAll()` 销毁所有界面，并通过 `Intent` 重定向回 `LoginActivity`。

```
© ActivityCollector.java  © BaseActivity.java  © LoginActivity.java ×  © MyDatabaseHelper.java  </> layout/ac

1  package com.example.my_campus.mycampus;
2
3  import android.content.Intent;
4  import android.content.SharedPreferences;
5  import android.os.Bundle;
6  import android.preference.PreferenceManager;
7  import android.widget.Button;
8  import android.widget.CheckBox;
9  import android.widget.EditText;
10 import android.widget.Toast;
11
12 public class LoginActivity extends BaseActivity {
13     private SharedPreferences pref;
14     private SharedPreferences.Editor editor;
15     private EditText accountEdit, passwordEdit;
16     private CheckBox rememberPass;
17
18     @Override
19     protected void onCreate(Bundle savedInstanceState) {
20         super.onCreate(savedInstanceState);
21         setContentView(R.layout.activity_login); // 需自行创建简单的登录XML布局
22
23         pref = PreferenceManager.getDefaultSharedPreferences(this);
24         accountEdit = findViewById(R.id.account);
25         passwordEdit = findViewById(R.id.password);
26         rememberPass = findViewById(R.id.remember_pass);
27         Button loginButton = findViewById(R.id.login);
28
29         boolean isRemember = pref.getBoolean(key: "remember_password", defValue: false);
30         if (isRemember) {
31             accountEdit.setText(pref.getString(key: "account", defValue: ""));
32             passwordEdit.setText(pref.getString(key: "password", defValue: ""));
33             rememberPass.setChecked(true);
34         }
35
36         loginButton.setOnClickListener(View v -> {
37             String account = accountEdit.getText().toString();
38             String password = passwordEdit.getText().toString();
39             // 简单模拟登录验证
40             if (account.equals("admin") && password.equals("123456")) {
41                 editor = pref.edit();
42                 if (rememberPass.isChecked()) {
43                     editor.putBoolean("remember_password", true);
44                     editor.putString("account", account);
45                     editor.putString("password", password);
46                 } else {
47                     editor.clear();
48                 }
49                 editor.apply(); // 持久化技术一：SharedPreferences 适用场景：简单配置
50
51                 Intent intent = new Intent(packageContext: LoginActivity.this, MainActivity.class);
52                 startActivity(intent);
53                 finish();
54             } else {
55                 Toast.makeText(context: LoginActivity.this, text: "账号或密码错误", Toast.LENGTH_SHORT).show();
56             }
57         });
58     }
59 }
60 }
```


④采用动态注册而非静态注册，保证了只有在 App 处于前台运行时才会响应下线通知，节省了系统资源，并符合最新的 Android 安全规范（标记了 RECEIVER_NOT_EXPORTED）。

3.数据持久化技术的综合应用及场景分析

①SharedPreferences(登录页密码记忆):

在 LoginActivity 中获取 SharedPreferences.Editor。当用户勾选“记住密码”并登录成功时，将账号和密码以键值对形式 putString 写入 XML 文件。

适用场景分析：极度轻量级，读取速度快。非常适合存储用户的个性化配置、简单的开关状态或简短的登录凭证。

②File 文件存储(缓存通知公告文本):

在 LeftFragment 中，利用 Context.openFileOutput 获得输出流，将一段较长的深大重要通知文本通过 BufferedWriter 直接写入手机内部存储的 txt 文件中。

```
DatabaseHelper.java <> layout\activity_main.xml <> sw600dp\activity_main.xml MainActivity.java LeftFragment.java
2
3 import android.content.Context;
4 import android.os.Bundle;
5 import android.view.LayoutInflater;
6 import android.view.View;
7 import android.view.ViewGroup;
8 import android.widget.Button;
9 import android.widget.TextView;
10 import androidx.fragment.app.Fragment;
11 import java.io.BufferedWriter;
12 import java.io.FileOutputStream;
13 import java.io.OutputStreamWriter;
14
15 <> public class LeftFragment extends Fragment {
16
17     @Override
18     public View onCreateView(LayoutInflater inflater, ViewGroup container, Bundle savedInstanceState) {
19         // 需自行创建 fragment_left.xml (包含一个通知文本框和一个下线测试按钮)
20         View view = inflater.inflate(R.layout.fragment_left, container, attachToRoot false);
21
22         Button testOfflineBtn = view.findViewById(R.id.btn_test_offline);
23         testOfflineBtn.setOnClickListener( View v -> {
24             ((MainActivity) getActivity()).simulateNetworkError();
25         });
26
27         // 数据持久化技术二：文件存储 (适用场景：缓存大段文本/重要通知)
28         saveNoticeToFile( inputText: "【重要通知】深大2025-2026学年校历发布...");
29
30         return view;
31     }
32
33     1 usage
34     private void saveNoticeToFile(String inputText) {
35         FileOutputStream out = null;
36         BufferedWriter writer = null;
37         try {
38             out = getActivity().openFileOutput( name: "notice_cache", Context.MODE_PRIVATE);
39             writer = new BufferedWriter(new OutputStreamWriter(out));
40             writer.write(inputText);
41         } catch (Exception e) {
42             e.printStackTrace();
43         } finally {
44             try {
45                 if (writer != null) writer.close();
46             } catch (Exception e) { e.printStackTrace(); }
47         }
48     }
```

适用场景分析：不对存储内容进行任何格式化处理。适合存储图片、音视频文件或大段的 JSON/HTML 原始文本流，但不方便进行特定字段的条件查询。

③SQLite 数据库(功能板块数据读取):

过程： 创建 MyDatabaseHelper 执行建表语句，初始化诸如“办事大厅”、“UOOC 课程”等带有分类（Category）的数据条目。在 RightFragment 中通过 getWritableDatabase().query() 方法，利用 Cursor 遍历查询结果并展示到界面上。

```
MyDatabaseHelper.java  <> layout/activity_main.xml  <> sw600dp/activity_main.xml  MainActivity.java  LeftFragment.java  v
1 package com.example.myapplication;
2
3 import android.content.Context;
4 import android.database.sqlite.SQLiteDatabase;
5 import android.database.sqlite.SQLiteOpenHelper;
6
7 2 usages
8 public class MyDatabaseHelper extends SQLiteOpenHelper {
9     1 usage
10     public static final String CREATE_SERVICE = "create table Service ("
11         + "id integer primary key autoincrement, "
12         + "category text, " // 如：教师事务、学生事务
13         + "name text");    // 如：办事大厅、UOOC课程
14
15     1 usage
16     private Context mContext;
17
18     1 usage
19     public MyDatabaseHelper(Context context, String name, SQLiteDatabase.CursorFactory factory, int version) {
20         super(context, name, factory, version);
21         mContext = context;
22     }
23
24     @Override
25     public void onCreate(SQLiteDatabase db) {
26         db.execSQL(CREATE_SERVICE);
27         // 初始化一些模拟数据
28         db.execSQL("insert into Service (category, name) values ('教师事务', '办事大厅')");
29         db.execSQL("insert into Service (category, name) values ('学生事务', 'UOOC课程')");
30         db.execSQL("insert into Service (category, name) values ('网上服务', '图书馆')");
31     }
32
33     10 usages
34     @Override
35     public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
36         // 升级逻辑
37     }
38 }
```

适用场景分析： 适用于具有复杂关系、结构化极强且数据量较大的场景。例如校园 App 中的通讯录、课程表或海量的新闻列表，方便使用 SQL 语句进行增删改查。

4.同一功能的多方案对比与优劣势分析

针对服务大厅网格界面的展示这一功能，在设计初期进行了两种方案的对比验证：

方案 A (传统嵌套流)： ScrollView 内部嵌套 ListView，ListView 的 Item 中再嵌套 GridView。

优势： 逻辑分层清晰，符合老版本 Android 的直觉开发模式。

劣势： 存在严重的滑动冲突问题（ScrollView 会导致内部 GridView 无法正确计算高度，只显示一行）。必须通过重写控件的 onMeasure 方法强行撑开高度，性能开销极大，且滑动卡顿。

方案 B (现代列表流/实验最终方案)： 使用 RecyclerView 搭配 GridLayoutManager。

优势： 彻底消灭了嵌套滑动冲突。只需重写 SpanSizeLookup 即可实现“标题栏占满整行（跨两列），具体服务项占据半行（占一列）”的复杂网格效果。配合自带的 ViewHolder 机制，内存复用率极高。

```

1 fragment.java  RightFragment.java  fragment_right.xml  fragment_left.xml  activity_login.xml
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:orientation="vertical"
6     android:padding="30dp"
7     android:gravity="center"
8     android:background="#FFFFFF">
9
10
11     <TextView
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="深圳大学统一身份认证"
15         android:textSize="24sp"
16         android:textStyle="bold"
17         android:textColor="#8A0025"
18         android:layout_marginBottom="40dp"/>
19
20     <EditText
21         android:id="@+id/account"
22         android:layout_width="match_parent"
23         android:layout_height="wrap_content"
24         android:hint="请输入校园卡号 (测试账号: admin)"
25         android:inputType="text" />
26
27     <EditText
28         android:id="@+id/password"
29         android:layout_width="match_parent"
30         android:layout_height="wrap_content"
31         android:hint="请输入密码 (测试密码: 123456)"
32         android:inputType="textPassword"
33         android:layout_marginTop="10dp"/>
34
35     <CheckBox
36         android:id="@+id/remember_pass"
37         android:layout_width="wrap_content"
38         android:layout_height="wrap_content"
39         android:layout_gravity="start"
40         android:layout_marginTop="10dp"
41         android:text="记住密码" />
42
43     <Button
44         android:id="@+id/login"
45         android:layout_width="match_parent"
46         android:layout_height="wrap_content"
47         android:layout_marginTop="20dp"
48         android:text="登 录"
49         android:textSize="18sp"
50         android:backgroundTint="#8A0025" />

```

```

1 MainActivity.java  LeftFragment.java  RightFragment.java  fragment_right.xml  fragment_left.xml
2 <?xml version="1.0" encoding="utf-8"?>
3 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent"
6     android:orientation="vertical"
7     android:padding="16dp"
8     android:background="#F5F5F5">
9
10     <TextView
11         android:layout_width="match_parent"
12         android:layout_height="wrap_content"
13         android:text="深大新闻与重要通知"
14         android:textSize="18sp"
15         android:textStyle="bold"/>
16
17     <TextView
18         android:layout_width="match_parent"
19         android:layout_height="wrap_content"
20         android:layout_marginTop="10dp"
21         android:text="文件缓存测试: 已将最新通知写入内部存储"
22         android:textColor="#666666"/>
23
24     <!-- 模拟网络断开, 触发强制下线广播的按钮 -->
25     <Button
26         android:id="@+id/btn_test_offline"
27         android:layout_width="match_parent"
28         android:layout_height="wrap_content"
29         android:layout_marginTop="30dp"
30         android:text="模拟网络异常 (触发强制下线)"
31         android:backgroundTint="#8A0025"/>
32 </LinearLayout>

```

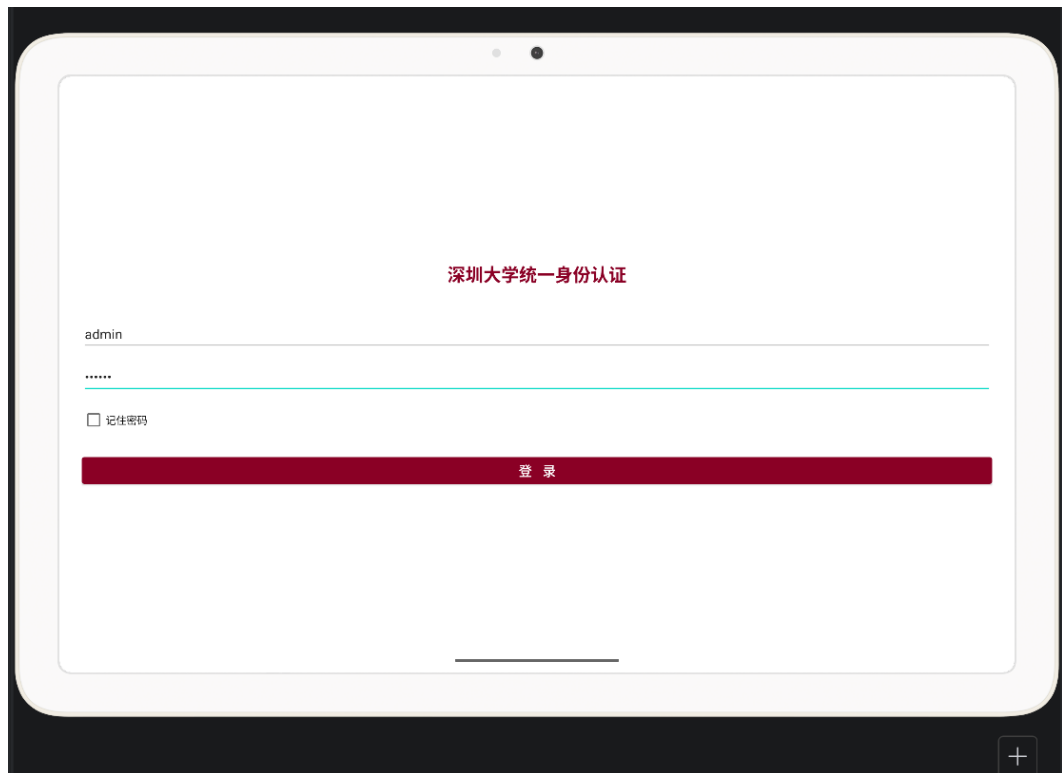
```

1 <?xml version="1.0" encoding="utf-8"?>
2 <ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:background="@drawable/bg" >
6
7     <LinearLayout
8         android:layout_width="match_parent"
9         android:layout_height="wrap_content"
10        android:orientation="vertical"
11        android:padding="16dp">
12
13        <!-- 顶部标题栏 -->
14        <TextView
15            android:layout_width="match_parent"
16            android:layout_height="wrap_content"
17            android:text="@string/title"
18            android:textColor="@color/white"
19            android:background="@color/black"
20            android:padding="8dp"
21            android:textStyle="bold"
22            android:gravity="center"/>
23
24        <GridLayout
25            android:layout_width="match_parent"
26            android:layout_height="wrap_content"
27            android:columnCount="2"
28            android:background="@color/white"
29            android:padding="16dp"
30            android:layout_marginBottom="16dp">
31
32            <TextView android:text="@string/item1" android:layout_width="80dp" android:layout_columnWeight="1" >
33            <TextView android:text="@string/item2" android:layout_width="80dp" android:layout_columnWeight="1" >
34            <TextView android:text="@string/item3" android:layout_width="80dp" android:layout_columnWeight="1" >
35            <TextView android:text="@string/item4" android:layout_width="80dp" android:layout_columnWeight="1" >
36            <TextView android:text="@string/item5" android:layout_width="80dp" android:layout_columnWeight="1" >
37        </GridLayout>
38
39        <!-- 中间内容区 -->
40        <TextView
41            android:layout_width="match_parent"
42            android:layout_height="wrap_content"
43            android:text="@string/content"
44            android:textColor="@color/white"
45            android:background="@color/black"
46            android:padding="8dp"
47            android:textStyle="bold"
48            android:gravity="center"/>
49
50        <GridLayout
51            android:layout_width="match_parent"
52            android:layout_height="wrap_content"
53            android:columnCount="2"
54            android:background="@color/white"
55            android:padding="16dp"
56            android:layout_marginBottom="16dp">
57
58            <TextView android:text="@string/item6" android:layout_width="80dp" android:layout_columnWeight="1" >
59            <TextView android:text="@string/item7" android:layout_width="80dp" android:layout_columnWeight="1" >
60            <TextView android:text="@string/item8" android:layout_width="80dp" android:layout_columnWeight="1" >
61            <TextView android:text="@string/item9" android:layout_width="80dp" android:layout_columnWeight="1" >
62            <TextView android:text="@string/item10" android:layout_width="80dp" android:layout_columnWeight="1" >
63        </GridLayout>
64
65        <!-- 底部按钮 -->
66        <TextView
67            android:layout_width="match_parent"
68            android:layout_height="wrap_content"
69            android:text="@string/button"
70            android:textColor="@color/white"
71            android:background="@color/black"
72            android:padding="8dp"
73            android:textStyle="bold"
74            android:gravity="center"/>
75
76        <GridLayout
77            android:layout_width="match_parent"
78            android:layout_height="wrap_content"
79            android:columnCount="2"
80            android:background="@color/white"
81            android:padding="16dp"
82            android:layout_marginBottom="16dp">
83
84            <TextView android:text="@string/item11" android:layout_width="80dp" android:layout_columnWeight="1" >
85            <TextView android:text="@string/item12" android:layout_width="80dp" android:layout_columnWeight="1" >
86            <TextView android:text="@string/item13" android:layout_width="80dp" android:layout_columnWeight="1" >
87            <TextView android:text="@string/item14" android:layout_width="80dp" android:layout_columnWeight="1" >
88            <TextView android:text="@string/item15" android:layout_width="80dp" android:layout_columnWeight="1" >
89        </GridLayout>
90
91        <!-- 底部提示 -->
92        <TextView
93            android:layout_width="match_parent"
94            android:layout_height="wrap_content"
95            android:text="@string/footer"
96            android:textColor="@color/white"
97            android:background="@color/black"
98            android:padding="8dp"
99            android:textStyle="bold"
100           android:gravity="center"/>
101
102        <GridLayout
103            android:layout_width="match_parent"
104            android:layout_height="wrap_content"
105            android:columnCount="2"
106            android:background="@color/white"
107            android:padding="16dp"
108            android:layout_marginBottom="20dp">
109
110            <TextView android:text="@string/item16" android:layout_width="80dp" android:layout_columnWeight="1" >
111            <TextView android:text="@string/item17" android:layout_width="80dp" android:layout_columnWeight="1" >
112            <TextView android:text="@string/item18" android:layout_width="80dp" android:layout_columnWeight="1" >
113            <TextView android:text="@string/item19" android:layout_width="80dp" android:layout_columnWeight="1" >
114            <TextView android:text="@string/item20" android:layout_width="80dp" android:layout_columnWeight="1" >
115        </GridLayout>
116
117        <!-- 底部数据加载提示 -->
118        <TextView
119            android:layout_width="match_parent"
120            android:layout_height="wrap_content"
121            android:text="@string/loading"
122            android:gravity="center"
123            android:layout_marginTop="16dp"
124            android:textColor="@color/white"/>
125
126        <!-- 底部加载更多按钮 -->
127        <TextView
128            android:id="@+id/tv_service_list"
129            android:layout_width="match_parent"
130            android:layout_height="wrap_content"
131            android:layout_marginTop="16dp"
132            android:layout_marginBottom="20dp"
133            android:background="@color/white"
134            android:padding="16dp"
135            android:textStyle="bold"
136            android:textColor="@color/black" />
137    </LinearLayout>
138 </ScrollView>

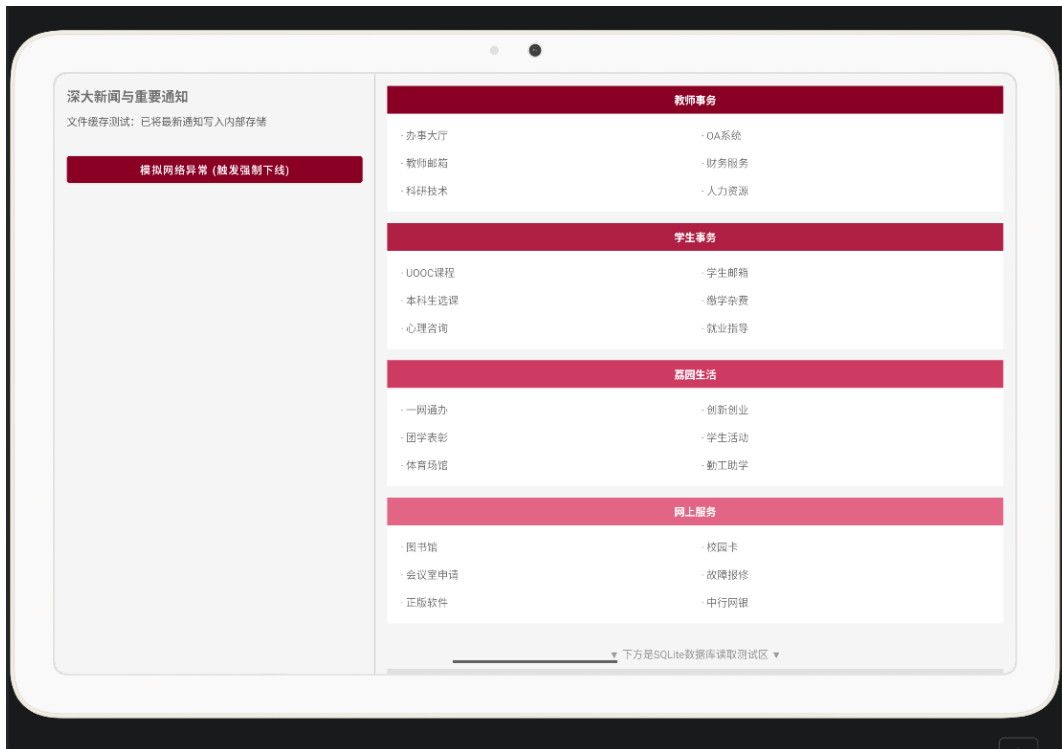
```

数据处理分析：

①运行 app，进入登录页



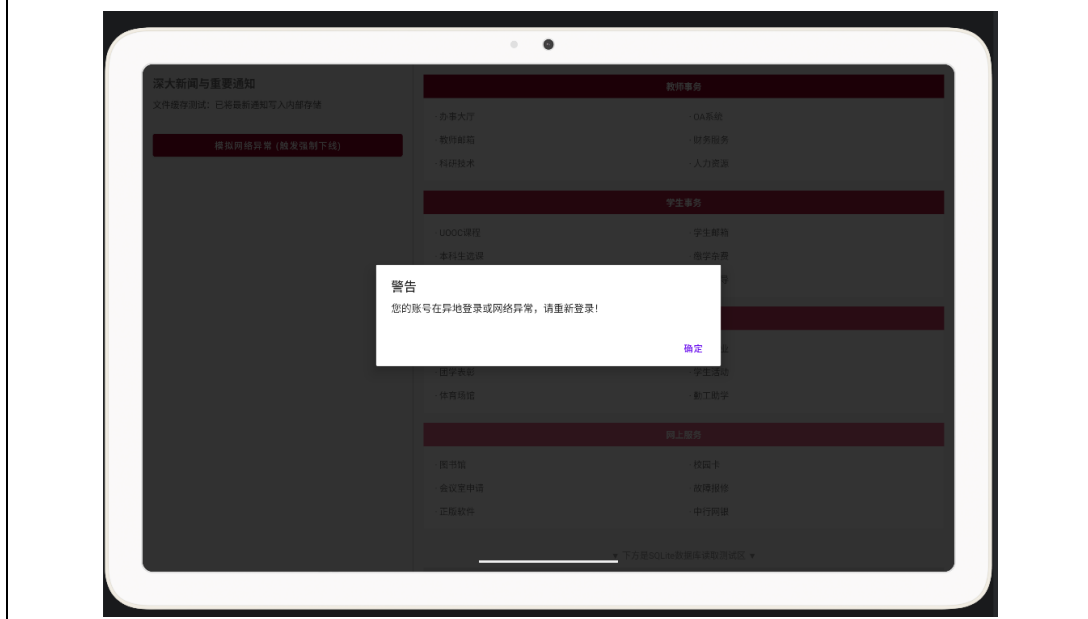
②登录后进入主页





左边为新闻区，右边为服务大厅各个板块。

③模拟强制下线



④数据持久化



若勾选“记住密码”，再次回到登录页会自动填写

深圳大学学生实验报告用纸

实验结论：

通过这次实验，我熟练掌握 Android Activity 生命周期管理、自定义 UI 布局开发、Fragment 碎片技术、全局广播机制及三种主流数据持久化方式。通过限定符布局实现手机 / 平板双端自适应，借助动态注册广播完成网络异常强制下线功能，运用 SharedPreferences、文件存储、SQLite 数据库三种持久化方案，分别实现密码记忆、通知缓存、服务数据管理，覆盖不同存储场景的技术选型需求。

对比传统嵌套布局与现代 RecyclerView 方案，实验中我采用的方案解决了滑动冲突、性能损耗等问题，采用更优的列表渲染方案提升界面流畅度；通过 ActivityCollector 统一管理活动栈，结合 BaseActivity 封装通用逻辑，优化代码结构与可维护性，符合 Android 开发规范。

本次实验将课堂理论与实际开发结合，深入理解 Android 组件通信、界面适配、数据存储与异常处理的核心逻辑。应用整体功能完整、界面规范、技术点覆盖全面，为后续复杂 Android 应用开发奠定了坚实基础。

指导教师批阅意见:

成绩评定:

指导教师签字：解为成
2026 年 5 月 日

备注:

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。